TaskFlow

Full-Stack Flask Todo Application

Project Report

Project	TaskFlow - Flask Todo App
Author	Muhammad Faizan Farooq
Role	Machine Learning and AI Engineer
Date	October 2026
Repository	github.com/faizanjutt473/TaskFlow-Flask-Todo-App
Technology	Python, Flask, Flask-SQLAlchemy, Flask-Login, SQLite, Bootstrap 5

Table of Contents

1. Introduction

2. Objectives

3. Key Features

4. Technology Stack

5. System Architecture

6. Database Design

7. Application Routes

8. Security Measures

9. Project Structure

10. User Interface

11. Installation and Usage

12. Testing

13. Future Enhancements

14. Conclusion

1. Introduction

TaskFlow is a full-stack web application for personal task management, built with the Flask micro-framework and a SQLite database. It lets people register an account, log in securely and manage their own to-do list from any device. Each user works in a private space: tasks created by one user are never visible to, or editable by, another user.

The project demonstrates the complete web development cycle: backend logic with routes and an ORM, relational database design, session-based authentication, server-side templating and a modern responsive interface built with Bootstrap 5, including a light and dark theme.

2. Objectives

- Build a complete, working full-stack application using Flask and SQLite.
- Implement secure user registration, login and logout with hashed passwords.
- Provide full CRUD (create, read, update, delete) operations on tasks for each authenticated user.
- Isolate data so that every user can access only their own tasks.
- Deliver a clean, professional and responsive user interface.
- Structure the project so that it is easy to read, maintain, extend and deploy.

3. Key Features

Area	Feature
Authentication	Register with unique email, password length check and confirmation; login with optional remember-me; logout; protected pages redirect to login.
Task management	Add, edit, delete, complete and undo tasks; title, description, priority and due date.
Priority system	Low, Medium and High priorities shown with colour-coded borders and badges.
Due dates	Dates are displayed in a readable format; overdue active tasks are highlighted in red.
Search and filter	Search by title and filter by All, Active or Completed.
Dashboard	Statistic cards for total, active and completed tasks plus a progress bar.
User interface	Responsive layout, dark mode with saved preference, flash messages, delete confirmation.

4. Technology Stack

Layer	Technology	Purpose
Language	Python 3.10+	Core application language
Web framework	Flask 3	Routing, request handling, templating integration
ORM	Flask-SQLAlchemy	Maps Python classes to database tables
Authentication	Flask-Login, Werkzeug	Session management and password hashing
Database	SQLite	Lightweight file-based relational database
Templates	Jinja2	Server-side HTML rendering
Frontend	HTML5, CSS3, JavaScript	Structure, styling and interactivity
UI framework	Bootstrap 5 and Icons	Responsive grid and components

5. System Architecture

The application follows a classic three-layer design. The presentation layer is the browser, which renders HTML, CSS and JavaScript. The application layer is Flask, which contains the authentication and task routes, the Flask-Login access control and the Jinja2 renderer. The data layer is a SQLite database accessed only through the SQLAlchemy ORM.

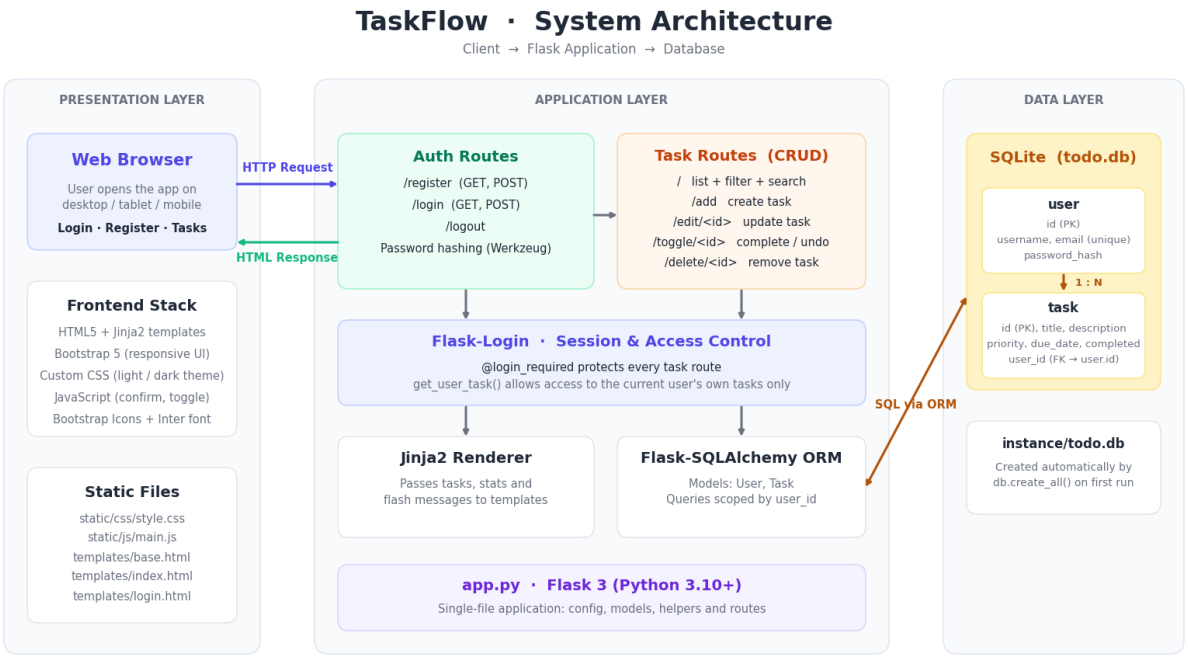


Figure 1: TaskFlow system architecture

Request flow

- The browser sends an HTTP request to a Flask route.
- For protected routes, Flask-Login checks the session; unauthenticated users are redirected to the login page.

- The route reads or writes data through SQLAlchemy models, always filtered by the current user's id.
- Jinja2 combines the data with an HTML template and Flask returns the response to the browser.

6. Database Design

The database has two tables connected by a one-to-many relationship: one user owns many tasks. Deleting a user also removes their tasks (cascade delete).

Table: user

Column	Type	Constraints
id	Integer	Primary key
username	String(50)	Not null
email	String(120)	Unique, not null
password_hash	String(255)	Not null (salted hash)
created_at	DateTime	Default: current UTC time

Table: task

Column	Type	Constraints
id	Integer	Primary key
title	String(150)	Not null
description	Text	Optional
priority	String(10)	Low, Medium or High (default Medium)
due_date	Date	Optional
completed	Boolean	Default False
created_at	DateTime	Default: current UTC time
user_id	Integer	Foreign key to user.id, not null

7. Application Routes

Route	Methods	Description	Login
/register	GET, POST	Create a new account with validation	No
/login	GET, POST	Authenticate and start a session	No
/logout	GET	End the session	Yes
/	GET	Dashboard: statistics, search, filters, task list	Yes
/add	POST	Create a task for the current user	Yes
/edit/<id>	GET, POST	Show and save changes to a task	Yes
/toggle/<id>	GET	Mark a task complete or active again	Yes
/delete/<id>	POST	Delete a task	Yes

8. Security Measures

- **Password hashing:** passwords are hashed with Werkzeug and never stored in plain text.
- **Access control:** every task route uses the @login_required decorator.
- **Data isolation:** a helper function loads a task only if it belongs to the logged-in user; otherwise a 404 error is returned.
- **Safe redirects:** the post-login next parameter is accepted only if it is a local path.
- **XSS protection:** Jinja2 auto-escapes all rendered values.
- **Safer deletion:** delete uses a POST request together with a browser confirmation.
- **Secret key:** should be loaded from an environment variable in production.

9. Project Structure

```
TaskFlow-Flask-Todo-App/
|-- app.py                  Flask app: config, models, routes
|-- requirements.txt        Python dependencies
|-- README.md               Project documentation
|-- LICENSE                 MIT License
|-- .gitignore              Ignored files
|-- docs/
|   |-- architecture.png    Architecture diagram
|   |-- preview.png         Application preview
|   `-- TaskFlow_Project_Report.pdf
|-- static/
|   |-- css/style.css       Custom styles and dark mode
|   `-- js/main.js          Delete confirm, alerts, theme toggle
`-- templates/
    |-- base.html           Base layout
    |-- index.html          Dashboard
    |-- edit.html           Edit task
    |-- login.html          Login page
    `-- register.html       Registration page
```

10. User Interface

The interface uses an indigo-to-violet gradient navbar, soft card layouts with rounded corners, colour-coded priorities and clear typography (Inter). The dashboard shows statistics at the top, a task creation form on the left and the searchable, filterable task list on the right. On small screens the layout stacks vertically for comfortable mobile use.

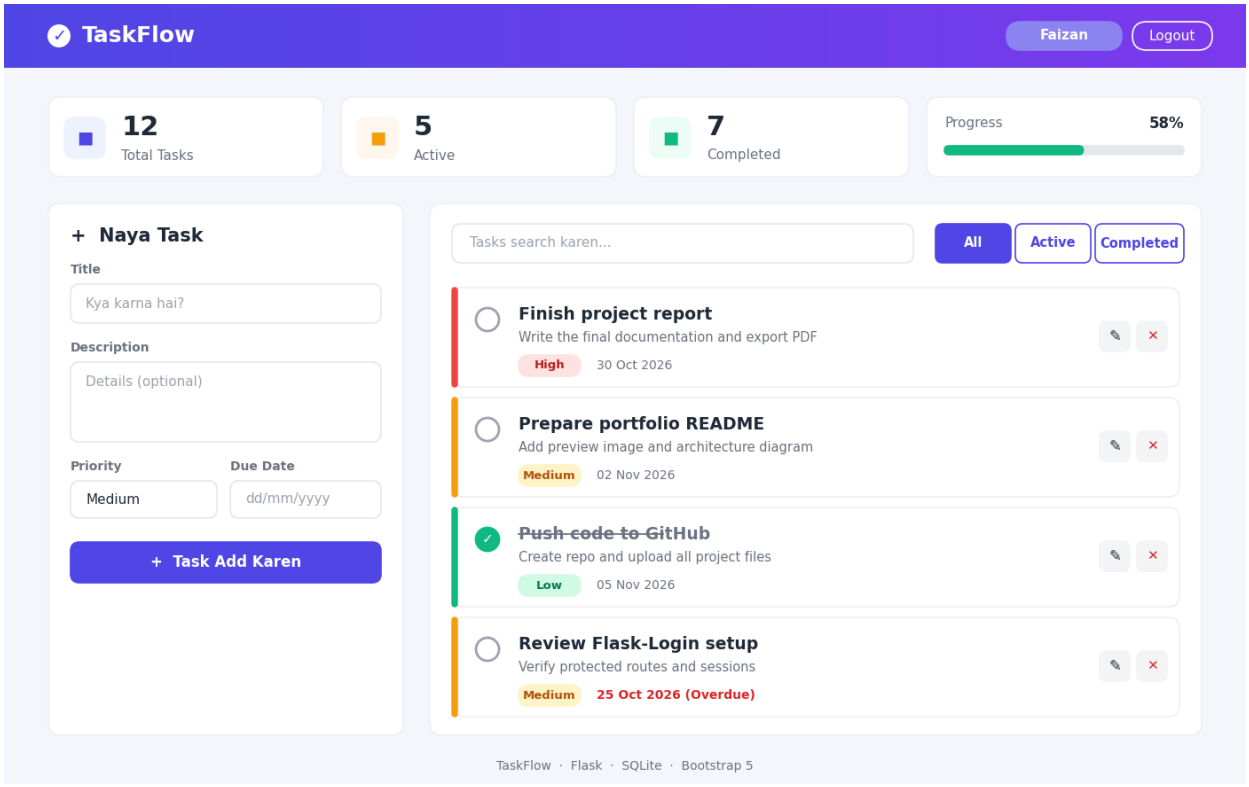


Figure 2: TaskFlow dashboard (design preview)

11. Installation and Usage

```
git clone https://github.com/faizanjutt473/TaskFlow-Flask-Todo-App.git
cd TaskFlow-Flask-Todo-App
python -m venv venv
venv\Scripts\activate          (Windows)
source venv/bin/activate       (macOS / Linux)
pip install -r requirements.txt
python app.py
```

Open http://127.0.0.1:5000 in a browser, register an account, log in and start adding tasks. The SQLite database file is created automatically inside the instance folder on the first run.

12. Testing

The application was tested manually using the following scenarios.

Test case	Expected result	Status
Register with valid data	Account created, redirect to login	Pass
Register with existing email	Error message shown	Pass
Register with mismatched passwords	Error message shown	Pass
Login with correct credentials	Redirect to dashboard	Pass
Login with wrong password	Error message shown	Pass
Open dashboard while logged out	Redirect to login page	Pass
Add, edit, complete, delete task	Dashboard updates correctly	Pass
Second user views the dashboard	Sees only their own tasks	Pass
Search and filter tasks	List shows matching tasks only	Pass
Switch dark / light mode	Theme changes and is remembered	Pass

13. Future Enhancements

- Task categories and tags.
- Email-based password reset and email verification.
- Drag-and-drop ordering and recurring tasks.
- REST API with token authentication.
- Pagination for large task lists.
- Automated tests with pytest.
- Docker support and deployment to a cloud platform.

14. Conclusion

TaskFlow shows how a small Flask project can be built into a complete, secure and good-looking web application. It covers authentication, database relationships, CRUD operations, server-side rendering and responsive design, and it provides a solid base that can grow with features such as categories, an API and cloud deployment.